DATA-260 Homework 1

Open-Source Package Vulnerability Reporting

Section 0- Personal Configuration and Domain

Name: Shrusti Shetty

SID4: 9004

PORT_BASE: 8004

PREFIX: s9004

SEED: 9004

VERIFY_SEED: 269004

DOMAIN_ID: 4

ASSIGNED DOMAIN: open-source package vulnerabilities

HARDWARE: Apple Silicon MacBook Pro

LOCAL MODEL: Ollama qwen3:8b

GitHub repository: https://github.com/shrustishetty-2401/data260-9004

TAGGED COMMIT HASH: cf5bd261f3e365c8b183e8984f801afcc04f038f

Repository structure and artifacts:

The project is maintained in the public GitHub repository

data260-9004.

The reports/hw01/ directory contains:

- **RUN_LOG.txt** — timestamped run information and console results.
- **raw/** — machine-readable JSON files for the 40 nondeterminism runs and per-turn token counts.
- **METRICS.md** — completed nondeterminism and token-accounting tables.

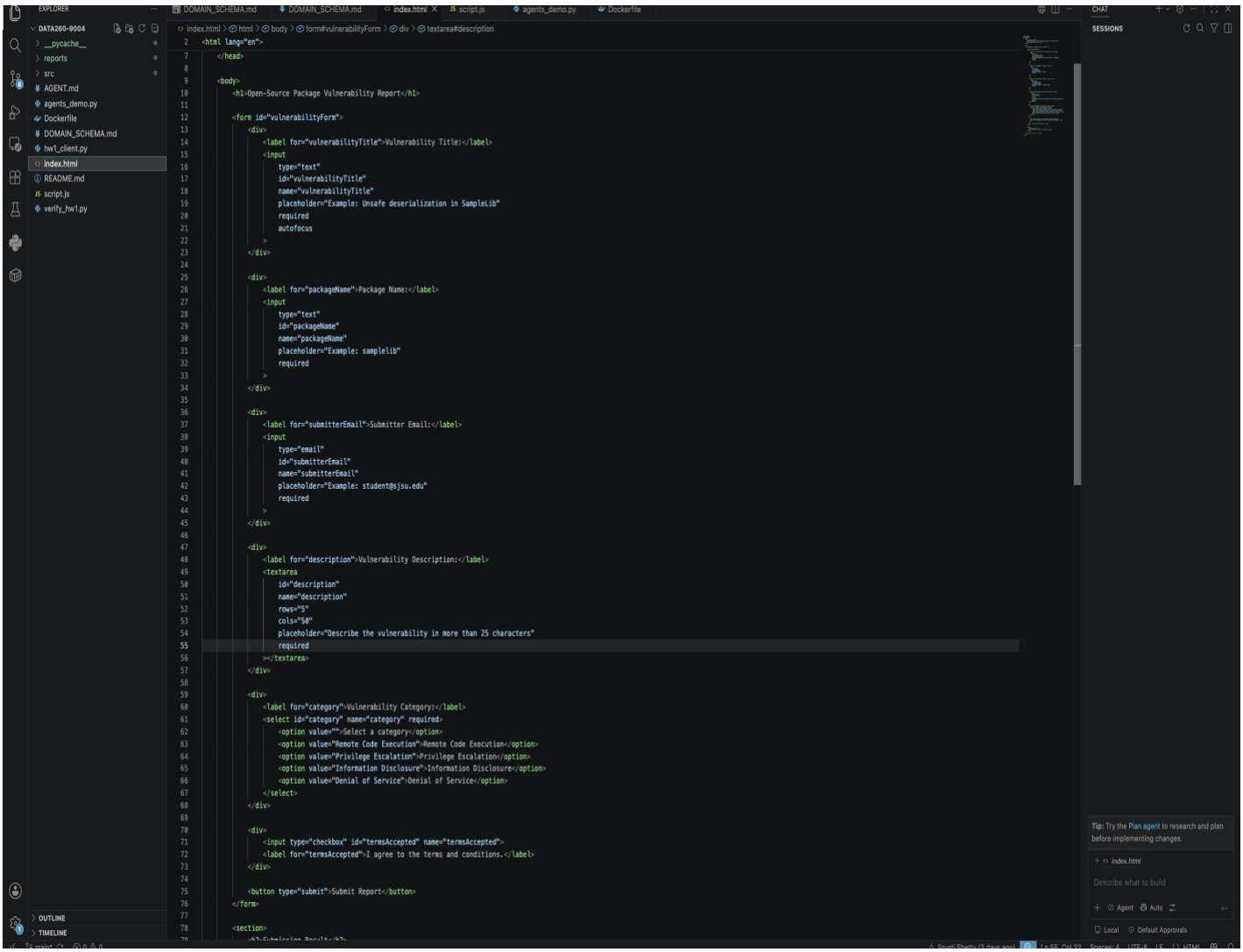
- **AI_USE.md** — answers describing AI assistance, verification, detection, and resulting changes.
- **report.pdf** — this final write-up with configuration values, screenshots, and answers.
- **README.md** — reproducible commands for running the application, agent pipeline, token demo, and verification.
- **verification.json** — output from the self-check script confirming that the required files and checks pass.

After all final edits are committed, the submitted repository state will be tagged hw1. The PDF submitted will be identical to reports/hw01/report.pdf at that tagged commit.

Part 1 — HTML:

I created an HTML page for reporting open-source vulnerabilities. The form includes fields for the vulnerability title, package name, submitter email, description, category, terms agreement, and report submission.

HTML Code Evidence:


EXPLORER
DATA260-9004
__pycache__
reports
src
AGENT.md
agents_demo.py
Dockerfile
DOMAIN_SCHEMA.md
hw1_client.py
index.html
README.md
script.js
verify_hw1.py
OUTLINE
TIMELINE
DOMAIN_SCHEMA.md
DOMAIN_SCHEMA.md
index.html
script.js
agents_demo.py
Dockerfile
index.html > html > body > form#vulnerabilityForm > div > textarea#description
<html lang="en">
</head>
<body>
<h1>Open-Source Package Vulnerability Report</h1>
<form id="vulnerabilityForm">

<label for="vulnerabilityTitle">Vulnerability Title:</label>
<input
type="text"
id="vulnerabilityTitle"
name="vulnerabilityTitle"
placeholder="Example: Unsafe deserialization in SampleLib"
required
autofocus
>

<label for="packageName">Package Name:</label>
<input
type="text"
id="packageName"
name="packageName"
placeholder="Example: samplelib"
required
>

<label for="submitterEmail">Submitter Email:</label>
<input
type="email"
id="submitterEmail"
name="submitterEmail"
placeholder="Example: student@sjsu.edu"
required
>

<label for="description">Vulnerability Description:</label>
<textarea
id="description"
name="description"
rows="5"
cols="50"
placeholder="Describe the vulnerability in more than 25 characters"
required
></textarea>

<label for="category">Vulnerability Category:</label>
<select id="category" name="category" required>
<option value="">Select a category</option>
<option value="Remote Code Execution">Remote Code Execution</option>
<option value="Privilege Escalation">Privilege Escalation</option>
<option value="Information Disclosure">Information Disclosure</option>
<option value="Denial of Service">Denial of Service</option>
</select>

<input type="checkbox" id="termsAccepted" name="termsAccepted">
<label for="termsAccepted">I agree to the terms and conditions.</label>

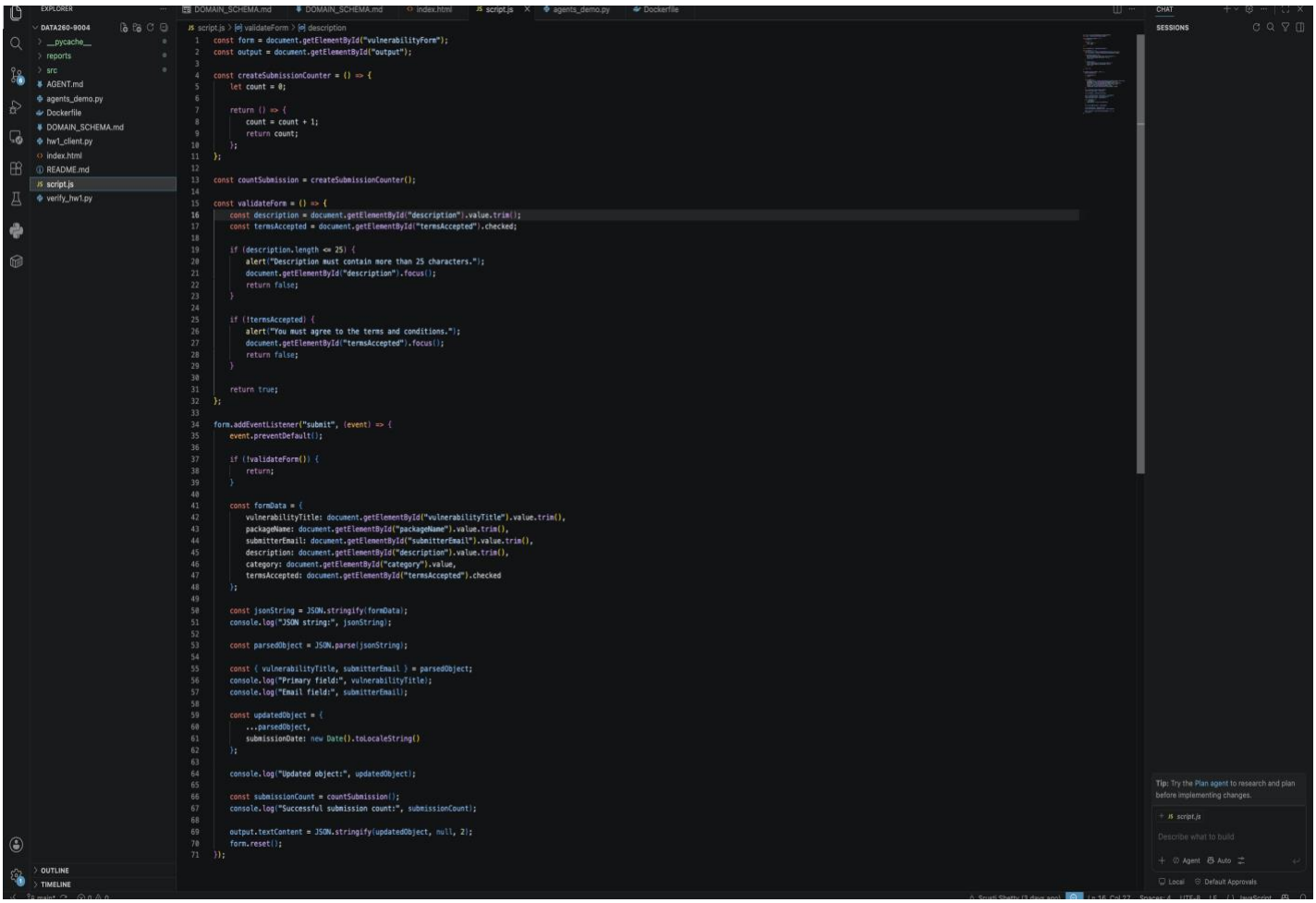
<button type="submit">Submit Report</button>
</form>
<section>
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
index.html
Describe what to build
Agent
Auto
Local
Default Approvals

Figure 1. Vulnerability-reporting form running locally in Docker at http://localhost:8004.

Part 2- JavaScript

The JavaScript validates the form before submission. It checks that the description contains more than 25 characters and that the terms checkbox is selected. After valid submission, it converts the form data to JSON and displays the result.

JavaScript Code Evidence:


EXPLORER
DATA260-9004
__pycache__
reports
src
AGENT.md
agents_demo.py
Dockerfile
DOMAIN_SCHEMA.md
hw1_client.py
index.html
README.md
script.js
verify_hw1.py
OUTLINE
TIMELINE
DOMAIN_SCHEMA.md
DOMAIN_SCHEMA.md
index.html
script.js
agents_demo.py
Dockerfile
script.js > validateForm > description
const form = document.getElementById("vulnerabilityForm");
const output = document.getElementById("output");
const createSubmissionCounter = () => {
let count = 0;
return () => {
count = count + 1;
return count;
};
};
const countSubmission = createSubmissionCounter();
const validateForm = () => {
const description = document.getElementById("description").value.trim();
const termsAccepted = document.getElementById("termsAccepted").checked;
if (description.length <= 25) {
alert("Description must contain more than 25 characters.");
document.getElementById("description").focus();
return false;
}
if (!termsAccepted) {
alert("You must agree to the terms and conditions.");
document.getElementById("termsAccepted").focus();
return false;
}
return true;
};
form.addEventListener("submit", (event) => {
event.preventDefault();
if (!validateForm()) {
return;
}
const formData = {
vulnerabilityTitle: document.getElementById("vulnerabilityTitle").value.trim(),
packageName: document.getElementById("packageName").value.trim(),
submitterEmail: document.getElementById("submitterEmail").value.trim(),
description: document.getElementById("description").value.trim(),
category: document.getElementById("category").value,
termsAccepted: document.getElementById("termsAccepted").checked
};
const jsonString = JSON.stringify(formData);
console.log("JSON string:", jsonString);
const parsedObject = JSON.parse(jsonString);
const { vulnerabilityTitle, submitterEmail } = parsedObject;
console.log("Primary field:", vulnerabilityTitle);
console.log("Email field:", submitterEmail);
const updatedObject = {
...parsedObject,
submissionDate: new Date().toLocaleString()
};
console.log("Updated object:", updatedObject);
const submissionCount = countSubmission();
console.log("Successful submission count:", submissionCount);
output.textContent = JSON.stringify(updatedObject, null, 2);
form.reset();
});
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
script.js
Describe what to build
Agent
Auto
Local
Default Approvals

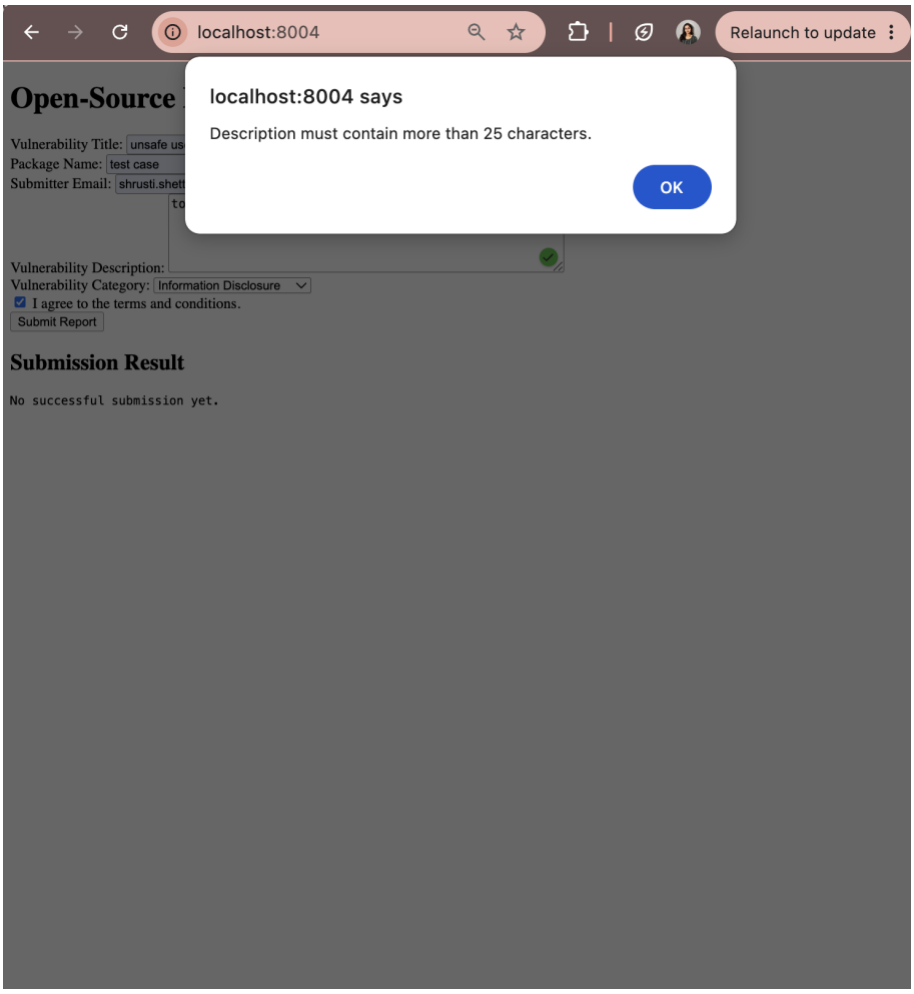


Figure 2. JavaScript validation alert for a description that is too short.

Figure 3. Successful form submission with JSON conversion and processed output

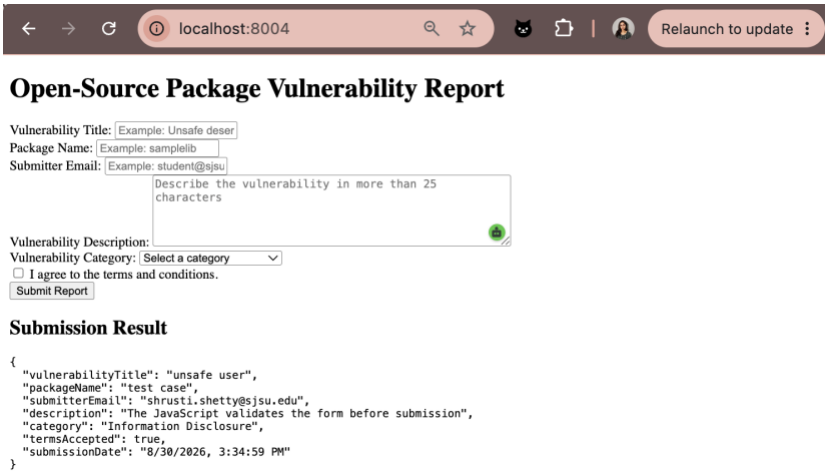
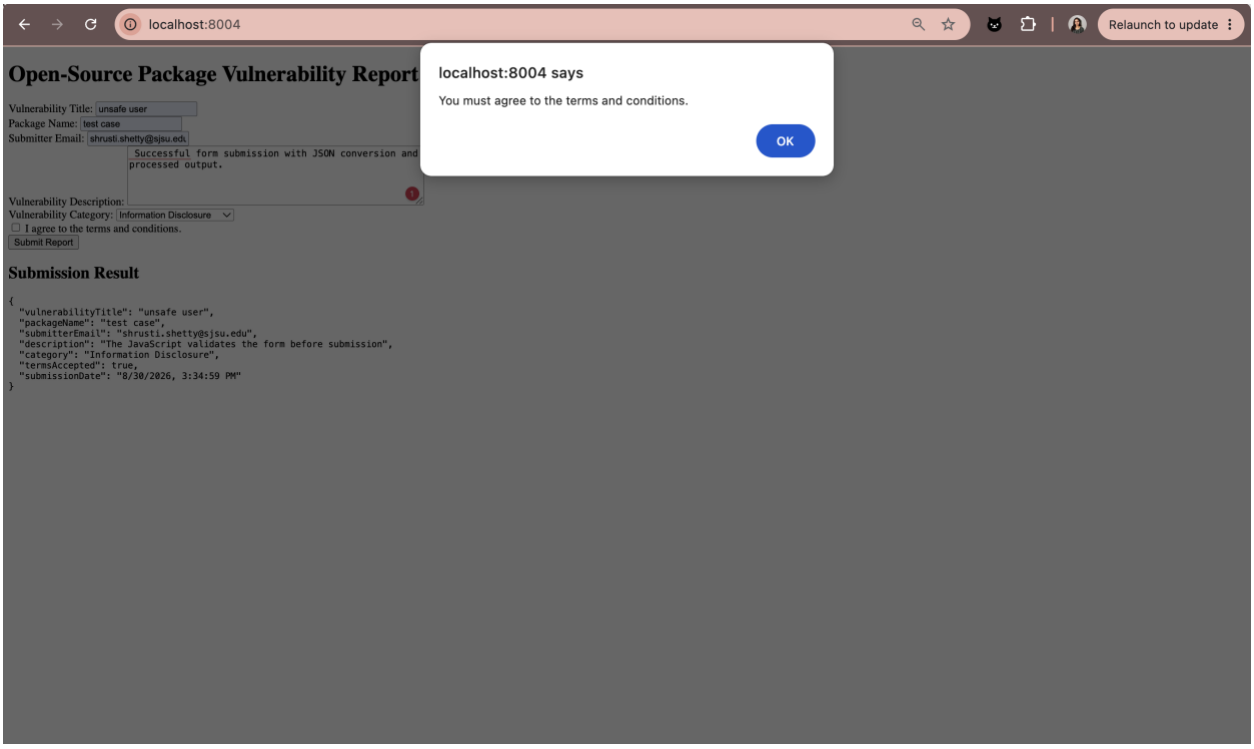


Figure 4. JavaScript validation alert when the terms checkbox is not selected.



The program uses destructuring to extract important form fields such as the vulnerability title and submitter email. It uses the spread operator to copy the submitted data and add a submissionDate field. A closure keeps track of the number of successful submissions without exposing the counter directly.

JavaScript Requirements Summary;

1. An arrow function validates that the description contains more than 25 characters.
2. The same validation checks that the terms-and-conditions checkbox is selected.
3. After successful submission, the form data is converted into a JSON string and logged.
4. Object destructuring extracts and logs the primary field and email.
5. The spread operator adds submissionDate with the current date and time.
6. A closure tracks and logs the number of successful submissions.

Part 3 — Docker Deployment

I containerized the web application with Docker. The image serves the HTML, CSS, and JavaScript files through a web server. I built the image and ran the container with port 8004 mapped to the container's HTTP port 80. The application was verified at http://localhost:8004.

docker build -t data260-9004-hw1 .

docker run --rm -d --name data260-hw1-local -p 8004:80 data260-9004-hw1

The local Docker evidence is shown in Figure 1.

AWS ECS Deployment:

Figure 5. AWS ECS service showing one running Fargate task and a successful deployment.

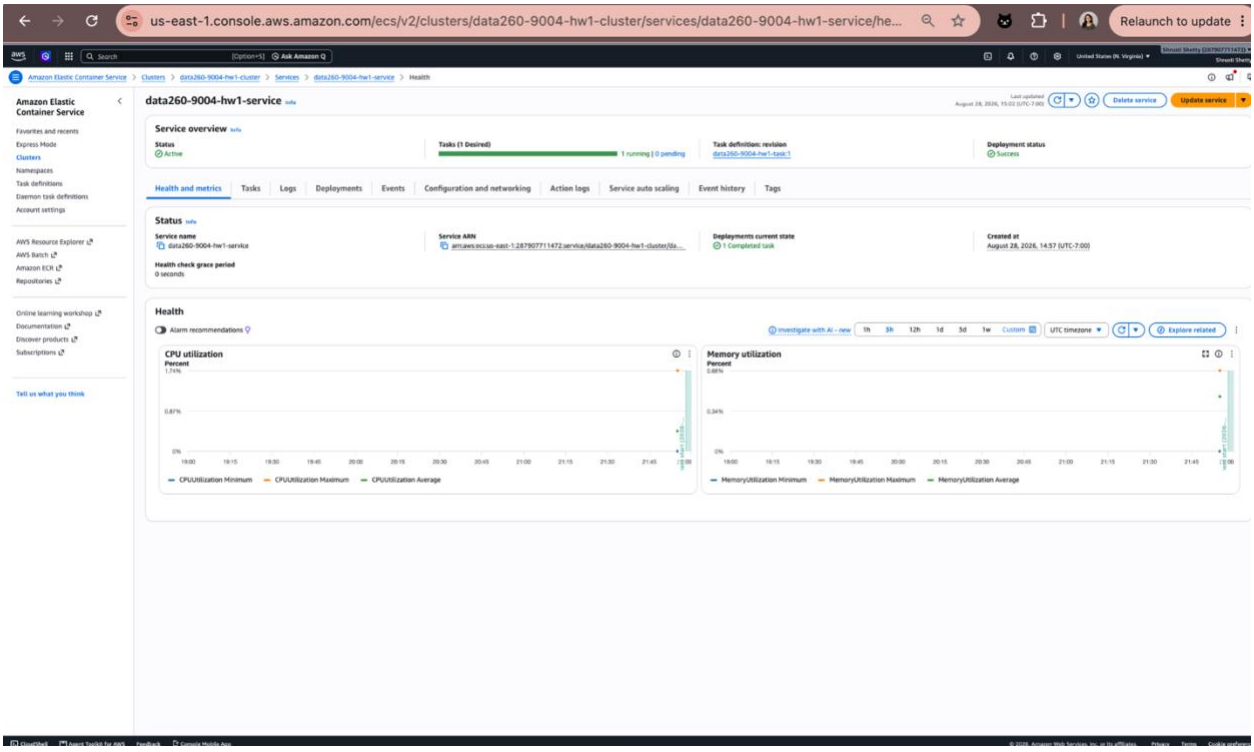


Figure 6. Vulnerability-reporting application running through the AWS ECS public IP.

The application was deployed to AWS ECS as a Fargate task in the us-east-1 region. The ECS service was active with one running task, and the deployment status was successful.

Agentic AI (Part 2)

The agentic AI pipeline uses three stages: a planner generates tags and a summary, a reviewer checks the result, and a finalizer prepares the approved output for publishing. The pipeline was run with the qwen3:8b model at temperature 0.0.

```
python agents_demo.py \
  --title "Open-source package vulnerability scanning" \
  --content "A dependency audit identifies outdated packages and recommends secure upgrades to reduce vulnerability risk." \
  --temperature 0.0
```

Figure 7. Agentic AI pipeline showing planning, review, finalization, and publish output.

```
(data260) srusti@Aashiks-MacBook-Pro data260-9004 % python agents_demo.py \
  --title "Open-source package vulnerability scanning" \
  --content "A dependency audit identifies outdated packages and recommends secure upgrades to reduce vulnerability risk." \
  --temperature 0.0
Tokens: input=96 output=38 total=134

Planner output:
{
  "tags": [
    "dependency audit",
    "package security",
    "vulnerability management"
  ],
  "summary": "A dependency audit identifies outdated packages and recommends secure upgrades to reduce vulnerability risk."
}
Tokens: input=130 output=79 total=209

Reviewer output:
{
  "tags": [
    "dependency audit",
    "package security",
    "vulnerability management"
  ],
  "summary": "A dependency audit identifies outdated packages and recommends secure upgrades to reduce vulnerability risk.",
  "changed": false,
  "explanation": "The Planner output meets the requirements: three distinct topical tags and a one-sentence summary of at most 25 words. No changes are needed."
}

Finalized output:
{
  "tags": [
    "dependency audit",
    "package security",
    "vulnerability management"
  ],
  "summary": "A dependency audit identifies outdated packages and recommends secure upgrades to reduce vulnerability risk."
}

Publish output JSON:
{"tags": ["dependency audit", "package security", "vulnerability management"], "summary": "A dependency audit identifies outdated packages and recommends secure upgrades to reduce vulnerability risk."}
(data260) srusti@Aashiks-MacBook-Pro data260-9004 %
```

The Planner creates three topical tags and a short summary from the input. The Reviewer checks whether the tags and summary meet the requirements and reports whether changes are needed. The Finalizer produces the approved result, which is then shown as publishable JSON.

Q1. Final tags:

dependency audit; package security; vulnerability management

Q2. Final summary:

A dependency audit identifies outdated packages and recommends secure upgrades to reduce vulnerability risk.

Q3. Did the Reviewer agent change anything?

No. The Reviewer found that the Planner already produced three topical tags and a summary within the required length, so no changes were needed.

The agents were implemented in Python 3.12.13 and used the local qwen3:8b model served through Ollama. The output was required to use strict JSON formatting. The agent logic was not hardcoded to this domain; the tags and summary were generated from the provided title and content.

1. The Planner reads the title and content and generates three topical tags plus a short summary.

2. The Reviewer checks the Planner output for the required number of tags, valid JSON, and summary length.

3. The Finalizer uses the reviewed result and prints the final Publish JSON.

Non-determinism Experiment (Part 3):

I used one fixed title and content input for all runs. I executed the experiment 40 times: 20 runs at temperature 0.7 and 20 runs at temperature 0.0. All 40 runs completed successfully.

METRIC	**TEMPERATURE 0.7**	**TEMPERATURE 0.0**
Successful runs	20	20
Distinct tag sets	5	1
Tags appearing in all 20 runs	None	dependency audit; package security; vulnerability management
Tags appearing in exactly one run	open source; package upgrades; security; security auditing; software security; vulnerability scanning	None

Latency table:

METRIC	**TEMPERATURE 0.7(ms)**	**TEMPERATURE 0.0(ms)**
p50	6946.11	7652.65
p95	7671.28	8587.59
p99	7816.91	8598.61

At temperature 0.7, the same input produced five distinct tag sets, showing greater variation between runs. At temperature 0.0, all 20 runs produced one stable tag set. Variation may be useful for exploratory brainstorming, but repeatable results are more appropriate for security and compliance classification.

The fixed input is stored in reports/hw01/cases/nondeterminism_input.json. The raw results and summary metrics are stored in reports/hw01/raw/nondeterminism_runs.json and reports/hw01/raw/nondeterminism_summary.json.

If two users submit the same input at temperature 0.7, they may receive different topical tags, while temperature 0.0 is expected to produce the same tags. Variation is acceptable during brainstorming, but not when consistent security or compliance classifications are required.

Part 4 — Model Adapter and Token Accounting,

The model adapter centralizes communication with the local language model. It sends the conversation history to Ollama, receives each response, and records input, output, and total token counts for every turn.

AGENT.md contains bullet-only instructions for verifying the project. The five-turn demo confirmed that the file passed verification and that the conversation history and token totals were tracked correctly.

CHECKPOINT	TURNS	INPUT TOKENS	OUTPUT TOKENS	TOTAL TOKENS	HISTORY LENGTH
After turn 3	3	479	132	611	1549
After turn 5	5	1129	213	1342	2256

Input tokens increased because the previous conversation history was included with each new request. The growing history allows the model to preserve context, but the model's context window limits how much conversation can eventually be retained.

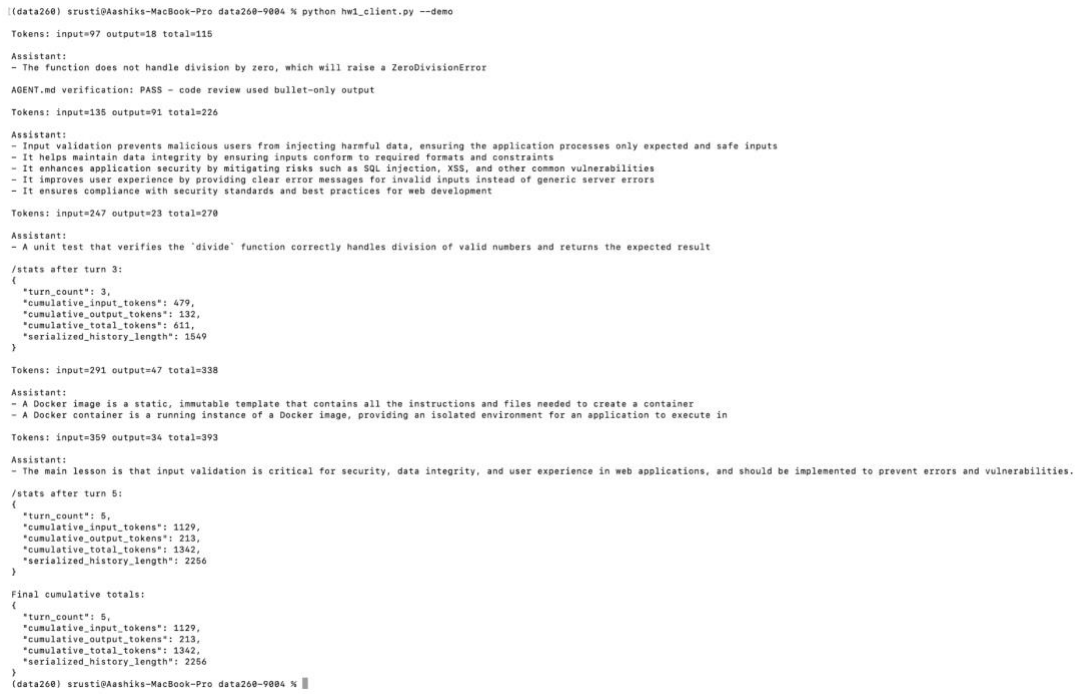


Figure 8. Five-turn conversation showing token counts, statistics checkpoints, and cumulative totals.

Each request includes the system instructions, the earlier conversation, and the new user message. Therefore, input-token counts grow as the conversation becomes longer. The model uses this context to answer consistently, but the context window limits the total amount of history that can be included.

Reproducibility and Verification:

The README.md file contains commands for building and running the Docker application, running the agent pipeline, running the token-accounting demo, and executing the verification script.

The self-check was run with:

python verify_hw1.py

The verification result was passed: true.

A system prompt contains the assistant’s instructions and rules, while a user message contains the user’s specific request or question. They have different roles in shaping the model’s response.

All model calls in the demo go through this adapter, and the /stats command reports statistics without changing the conversation history.